

Take-Home Assignment: Question-Answering Service

Time-box: This is designed to take a focused 8-12 hours. Please don't spend your whole week on it - we're more interested in your judgement under a constraint than in a polished product. If you run short on time, leave clear notes on what you'd do next instead of cutting corners silently.

Sent: Friday · **Due:** end of day Thursday

Context

We're building an AI layer for an enterprise platform. The foundation of everything we want it to do - across finance, HR, CRM, and more - is the same: take a user's natural-language question, find the relevant information inside the company's own documents, answer it **accurately**, and **never make things up**. Everything else is built on top of that core. This assignment asks you to build a small version of exactly that core.

The Task

You're given a small set of business documents (attached: an employee handbook, a financial summary, a vendor contract, meeting notes, a CRM record, and an invoice).

Build a small **service** that:

1. **Ingests** the provided documents.
2. **Answers natural-language questions** asked over them (e.g. "How many PTO days can I carry into next year?", "What was our Q2 operating profit?", "When does the Acme contract auto-renew, and what notice is required to stop it?").
3. **Grounds every answer in the source** - each answer must cite which document (and ideally which part of it) the answer came from, so a user can verify it.
4. **Says "I don't know" when the answer isn't in the documents**. Some questions deliberately have no answer in the body. Confidently inventing an answer is worse than admitting the information isn't there.

It can be an HTTP API or a command-line interface - your choice. It does **not** need a front end. Treat it as something that could plausibly grow into a production service, not a one-off notebook.

The Part that Matters Most: Measuring Quality

Anyone can wire a model to some documents and get answers that *look* right. The harder and more important question is: **how do you know the answers are actually good?**

So, in addition to the service, we want you to:

- Create a small **evaluation set** of question/expected-answer pairs (aim for 12-15), including some questions whose answer is *not* in the documents.
- Build a **repeatable way to score** your service against that set - automated if you can, even if simple. Tell us what you've measuring and why.

- **Report the results.** When does it do well? Where does it fail? When it gives a wrong answer, can you tell whether the problem was *retrieval* (it pulled the wrong source) or *generation* (it had the right source but answered badly)?

We care more about thoughtful measurement than about a high score. A system that's 80% correct with an honest, well-reasoned eval beats one that claims 100% with no way to prove it.

Deliverables

1. **The code**, in a Git repo (or a zip), that we can run from a clear README.
2. A **README** with: how to run it, the key design decisions you made and why, and anything you'd do differently with more time.
3. Your **evaluation set and results**, with a short discussion of the failure modes you found.

Ground Rules

- Use any language, libraries, framework, or model you like. (For reference, our AI layer will be in the Python ecosystem - but build it however you work best, and tell us why you chose what you chose.)
- Using an LLM API is expected. We'll provide an API key, ask us.
- You're encouraged to keep it simple. Reach for complexity only where it earns its place.

Stretch (only if you have time - not required)

Our real platform is **multi-tenant**: many companies share the system, and one of our company's documents must *never* surface in another company's answers. If you want to show how you'd think about that, sketch (in writing) how you'd keep tenant A's documents isolated from tenant B's at retrieval time. A paragraph of clear thinking here is worth more than a half-built feature.

How We'll Evaluate It

We'll look at:

- **Accuracy & grounding** - are answers correct and genuinely tied to the source?
- **Honesty** - does it admit when it doesn't know, instead of hallucinating?
- **Evaluation rigor** - this is the heaviest factor. Do you have a real way to know if it's good, and can you reason about *why* it fails?
- **Engineering** - is it structured like something that could become a real service?
- **Communication** - can you explain your choices and trade-offs clearly?

The eval section is the part we care about most, and a smaller working system with a thoughtful eval beats a bigger system with none.

Good luck :) - We're genuinely interested in how you think, not just what you ship.